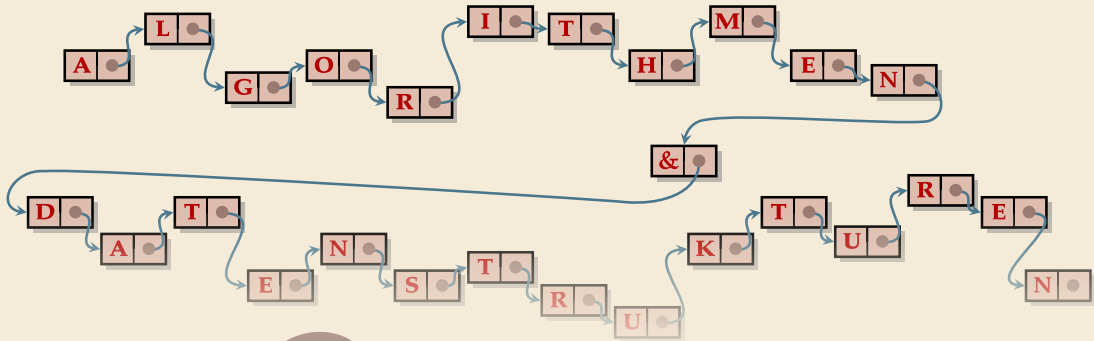




8

Dynamische Listen

Algorithmen & Datenstrukturen · Sommersemester 2026

Prof. Dr. Sebastian Wild

8 Dynamische Listen

- 8.1 Abstract Data Types
- 8.2 Linear Listen
- 8.3 Stacks & Queues
- 8.4 Array-basierte Listen
- 8.5 Verkettete Listen
- 8.6 Doubling Arrays
- 8.7 Java Collections Framework

Dynamische Datenstrukturen

Bisher: im Wesentlichen alles Arrays!

- ▶ Sortieren  außer Union-Find!
- ▶ Suchen
- ▶ Subsequence Sums
- ▶ 2D-Felder

Jetzt: Dynamische Strukturen!

~> Unit 8: Sequenzen von Elementen („Listen“)

8.1 Abstract Data Types

Abstract Data Types

Für algorithmische Probleme, starten mit Gegeben-Ziel-Spezifikation

► Sortieren

Gegeben: Array $A[0..n)$ von n Objekten, Sortierkriterium

Ziel: Array so permutieren, dass Objekte aufsteigend sortiert sind.

Abstrakter Datentyp (ADT) ist das Pendant für Datenstrukturaufgaben

- spezifiziert Operationen (Queries, Updates) auf Daten mit Semantik
- Bei Queries: Angabe welcher Rückgabewert produziert werden soll
- Bei Updates: Angabe, wie sich der Inhalt der Datenstruktur semantisch ändert

≈ Java Interface

≠ Implementierung/Datenstruktur: ADT gibt nur an *was* passieren soll, nicht *wie*

Beispiel

ADT *Union-Find* (Disjunkte Mengen)

- ▶ Operationen: $\text{construct}(n)$, $\text{union}(p, q)$, $\text{find}(p)$
- ▶ Semantik
 - ▶ construct gibt Größe n des Universums fest vor (unveränderlich)
 - ▶ Zu jedem Zeitpunkt haben wir eine Partition $S_0, \dots, S_{k-1}$ von $\{0, \dots, n-1\}$ für ein k .
 - ▶ Initial ist $k = n$ und $S_i = \{i\}$
 - ▶ Query $\text{find}(p)$ gibt das eindeutige i zurück mit $p \in S_i$
 - ▶ Update $\text{union}(p, q)$: mit $i = \text{find}(p)$ und $j = \text{find}(q)$ ersetze S_i und S_j durch $S_i \cup S_j$.
- ▶ Java Interface

```
1 public interface UnionFind {  
2     // UnionFind(int n);  
3     // Konstruktoren nicht Teil von Interfaces  
4  
5     /** gibt das eindeutige i zurück mit  $p \in S_i$  */  
6     int find(int p);  
7  
8     /** mit  $i = \text{find}(p)$  und  $j = \text{find}(q)$   
9         ersetze  $S_i$  und  $S_j$  durch  $S_i \cup S_j$  */  
10    void union(int p, int q);  
11 }
```

Java Generics & Collections

Weit-verbreitete Datenstrukturen: **Container / Collections**

- ▶ bestehen aus einer Sammlung von Objekten
- ▶ Objekte für Container eine “black box” $\rightsquigarrow$ egal was drin steckt
- ▶ typische Anwendung: erlauben nur „sortenreine“ Elemente
 - ▶ Liste von Strings
 - ▶ Menge von Dateien
 - ▶ Matrix von ganzen Zahlen

$\rightsquigarrow$ möchten „sortenrein“ spezifizieren können, ohne vorher „Sorte“ festzulegen

Analog zu `int[]`, `String[]`, etc.!

Typparameter! (in Java: *Generics*)

- ▶ Klassen und Interfaces in Java können Typparameter haben
- ▶ wird in Klasse verwendet wie ein konkreter Klassenname
- ▶ erlaubt aber (bei Instanziierung) das Einsetzen beliebiger Klassen
- ▶ Beispiel `Comparator<T>`

8.2 Linear Listen

Linear Listen

Fundamentaler Container: *ADT Lineare Liste (Dynamische Sequenz)*

- Semantik: zu jedem Zeitpunkt ist L eine Sequenz $a_1, a_2, \dots, a_n$ von Elementen.

```
1 interface LinearList<Elem, Position extends LinearList.ListPosition> {  
2     Position start();  
3     Position end();  
4     Position next(Position p);  
5     Position previous(Position p);  
6  
7     void insert(Position p, Elem x);  
8     void delete(Position p);  
9     Elem get(Position p);  
10    void set(Position p, Elem x);  
11  
12    /** Represents a position of an element in the list */  
13    interface ListPosition { }  
14  
15    // default methods (siehe unten)  
16 }
```

- zweiter Typparameter Position hier nötig, weil die Semantik von Position von Implementierung abhängt
 ~> dazu gleich mehr!

Linear Listen – Positionen

Für $L = a_1, \dots, a_n$ können wir als Positionen einfach Indices $p \in \mathbb{N}_{\geq 1}$ angeben

- ▶ $L.start() := 1$ erstes Element
- ▶ $L.end() := n + 1$ *hint*er letztem Element
- ▶ $L.next(p) := \begin{cases} p + 1 & \text{falls } p \neq L.end(), \\ \text{Exception} & \text{sonst.} \end{cases}$
- ▶ $L.previous(p) := \begin{cases} p - 1 & \text{falls } p \neq L.start(), \\ \text{Exception} & \text{sonst.} \end{cases}$

(Positionen erscheinen unnötig? $\rightsquigarrow$ siehe unten)

Linear Listen – Operationen

Eigentliche Operationen $L = a_1, a_2, \dots, a_n$ und $p \in [1..n + 1]$:

$$\blacktriangleright L.get(p) := \begin{cases} a_p & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

- $\blacktriangleright L.set(p, x)$ wirft Exception falls $p = L.end()$ und terminiert sonst normal.

$$\text{Danach gilt } L = \begin{cases} a_1, \dots, a_{p-1}, x, a_{p+1}, \dots, a_n & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

- $\blacktriangleright$ Nach $L.insert(p, x)$ gilt $L = a_1, \dots, a_{p-1}, x, a_p, a_{p+1}, \dots, a_n$.

Beachte: $p = L.end()$ erlaubt! $\rightsquigarrow L.insert(p, x)$ hängt x ans Ende von L an.

- $\blacktriangleright L.delete(p)$ wirft Exception falls $p = L.end()$ und terminiert sonst normal.

$$\text{Danach gilt } L = \begin{cases} a_1, \dots, a_{p-1}, a_{p+1}, \dots, a_n & \text{falls } p \neq L.end(), \\ \text{undefiniert} & \text{sonst.} \end{cases}$$

Wozu Positionen?

Es scheint ganz und gar unnötig kompliziert, dass wir Operationen für Positionen haben.

Wir könnten doch einfach direkt den Index p übergeben; was soll all der Ärger?

Jein . . . wir haben nämlich eine Subtilität unterschlagen:

*Was passiert bei **insert**(L, p) / **delete**(L, p) mit der Position p ?*

Antwort: Implementierungen von Linearen Listen können Realisierungen wählen!

Index-basierte Positionen

- ▶ entspricht obigem Modell einer mathematischen Sequenz
- ▶ $p \in \mathbb{N}_{\geq 1}$ stets das p -te Element vom Anfang auch nach **insert/delete**

↪ Updates links von p ändern Wert von $\text{get}(p)$!

Pointer-basierte Positionen

- ▶ entspricht einer verketteten Liste (Index vielleicht gar nicht bekannt)
- ▶ Position verweilt auf „ihrem“ Element auch nach **insert**

↪ Updates anderswo ändern Wert von $\text{get}(p)$ **nicht**

Abgeleitete Operationen

Einige Hilfsoperationen für Lineare Listen können wir implementieren, ohne zu wissen, welche Listen-Implementierung wir vorliegen haben!

Ultimativer wiederverwendbarer Code

- ▶ `L.append(x)`: hängt `x` ans Ende von `L` an.

```
1 // public interface LinearList...
2     default void append(Elem x) {
3         insert(isEmpty() ? start() : end(), x);
4     }
```

- ▶ `L.prepend(x)`: fügt `x` am Anfang (vor allen existierenden Elementen) von `L` an.

```
1     default void prepend(Elem x) { insert(start(), x); }
```

- ▶ `L.isEmpty()`: liefert `true`, wenn `L` keine Elemente enthält und `false` sonst.

```
1     default boolean isEmpty() { return start().equals(end()); }
```

Abgeleitete Operationen [2]

- ▶ `L.firstOccurrence(x)`: erste Position in `L`, an der `x` in der Liste vorkommt bzw. `L.end()` falls `x` gar nicht in der Liste vorkommt.
- ▶ `L.firstOccurrenceAfter(p,x)`: erste Position in `L` hinter `p` (inklusive `p` selbst), an der `x` in der Liste vorkommt bzw. `L.end()` falls `x` dort nicht in der Liste vorkommt.

```
1  default Position firstOccurrence(Elem x) {  
2      return firstOccurrenceAfter(start(), x);  
3  }  
4  default Position firstOccurrenceAfter(Position p, Elem x) {  
5      for (Position cur = p; !cur.equals(end()); cur = next(cur))  
6          if (get(cur).equals(x)) return cur;  
7      return end();  
8  }  
9  // } end of interface LinearList
```

- ▶ Beachte: Können hier nur die lineare Suche verwenden.

8.3 Stacks & Queues

Eingeschränkte ADTs

Bevor zu Implementierungen von Linearen Listen kommen, betrachten wir zwei Spezialfälle: Stacks und Queues

- ▶ Da beide nur Teile der Operationen von Linearen Listen brauchen ist jede Implementierung von Linearen Listen auch ein Stack und eine Queue

⇒ *brauchen keine neuen Implementierungen*

Eingeschränkte ADTs trotzdem oft sinnvoll:

- ▶ Kommunizieren zu anderen Entwickler:innen, dass hier nur Spezialfall gebraucht
 - ⇒ mehr Information über Ihren Code
- ▶ (manchmal) bessere Implementierung für einfachere Anforderung möglich

Stacks



ADT *Stack* (*Stapel*, *Kellerspeicher*)

- ▶ `top()`: Gib oberstes Element zurück.
Stack unverändert
- ▶ `push(x)`: Lege *x* oben auf den Stack
(Einfügen).
- ▶ `pop()`: Entferne oberstes Element und gib es
zurück
- ▶ `empty()`: true gdw. Stack leer

```
1 public interface Stack<Elem> extends Iterable<Elem> {  
2     void push(Elem x);  
3     Elem pop();  
4     Elem top();  
5     boolean empty();  
6 }
```

Queues

ADT *Queue* (Warteschlange):

- ▶ `enqueue( $x$ )`: Füge x am Ende der Queue ein.
- ▶ `dequeue()`: Entferne Element am Beginn der Queue und gib es zurück.



Bags

Manchmal wollen wir kommunizieren, dass die Reihenfolge der Elemente völlig egal ist.

ADT *Bag/Collection (Sack)*

- ▶ `insert( $x$ )`: Füge x zur Bag hinzu
- ▶ `delAny()`: Entferne *beliebiges* Element aus Bag und gib es zurück.
(Reihenfolge unspezifiziert)
- ▶ Java's `java.util.Collection`



Meist realisiert als Stack oder Queue, aber Reihenfolge soll für Benutzer der Bag keinen Unterschied machen.

Deque

ADT *Deque* (Double-Ended Queue)

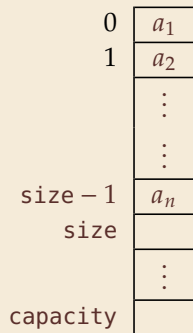
- ▶ Einfügen und Entfernen von Elementen an beiden Enden erlaubt

8.4 Array-basierte Listen

Array-basierte Listen

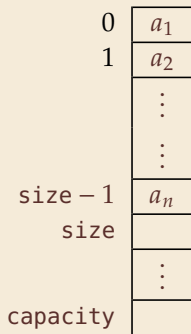
Idee: Speichere Elemente der Linearen Liste in einem Array.

- ▶ Array-basierte Lineare Listen bleiben am nächsten an mathematischer Sequenz
- ▶ Array hat feste Größe $\rightsquigarrow$ im Konstruktor fest gewählt.



```
1 public class FixedSizeArrayList<Elem>
2     implements LinearList<Elem, FixedSizeArrayList.Position> {
3     private final int capacity;
4     private final Object[] elements;
5     private int size = 0;
6
7     public FixedSizeArrayList(int capacity) {
8         this.capacity = capacity;
9         elements = new Object[capacity];
10    }
11    /** Represents an array index */
12    public static class Position implements LinearList.ListPosition {
13        private final int index;
14        public Position(int index) { this.index = index; }
15        public boolean equals(Object o) {
16            return index == ((IndexPosition) that).index;
17        }
18        public int hashCode() { return index; }
19    }
20    // methods
21 }
```

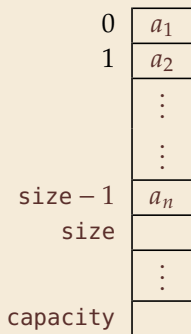
Array-basierte Listen – Günstige Operationen



```
1 //public class FixedSizeArrayList<Elem> {  
2     public Position start() { return new Position(0); }  
3     public Position end() { return new Position(size); }  
4     public Position next(Position p) {  
5         if (p.index == size) throw new IndexOutOfBoundsException();  
6         return new Position(p.index+1);  
7     }  
8     public Position previous(Position p) {  
9         if (p.index == 0) throw new IndexOutOfBoundsException();  
10        return new Position(p.index-1);  
11    }  
12    public Elem get(Position p) { return elements[p.index]; }  
13    public void set(Position p, Elem x) { elements[p.index] = x; }  
14    // more methods  
15 }
```

- alle diese Operationen benötigen nur $O(1)$ Laufzeit

Array-basierte Listen – Teure Operationen



```
1 //public class FixedSizeArrayList<Elem> {  
2     public void insert(Position p, Elem x) {  
3         if (size >= capacity) throw new CapacityExceededException();  
4         for (int j = size - 1; j >= p.index; --j)  
5             elements[j + 1] = elements[j];  
6         elements[p.index] = x; ++size;  
7     }  
8     public void delete(Position p) {  
9         if (p.equals(end())) throw new IndexOutOfBoundsException();  
10        for (int j = p.index; j <= size-2; ++j)  
11            elements[j] = elements[j+1];  
12        elements[size-1] = null; // avoid loitering  
13        --size;  
14    }  
15 //}
```

- können zwar direkt auf Position p zugreifen, müssen aber alle Elemente rechts von p um eine Position verschieben

↪ Worst-Case $O(n)$ Laufzeit

- ⚡ keine allgemeine Implementierung: insert sollte keine Exception werfen

Ringpuffer

Für Queues (und ähnliche ADTs) mit Updates nur am linken/rechten Rand können wir uns den Verschiebeaufwand sparen!

8.5 Verkettete Listen

DIY Memory Management

8.6 Doubling Arrays

Doubling Arrays

8.7 Java Collections Framework

Iteratoren für unsere Lineare Liste

```
1 interface LinearList<Elem, Position extends LinearList.ListPosition> extends Iterable<Elem> {
2     // ...
3     default Iterator<Elem> iterator() {
4         return new Iterator<Elem>() {
5             Position cur = start();
6             public boolean hasNext() { return !cur.equals(end()); }
7             public Elem next() {
8                 if (!hasNext()) throw new NoSuchElementException();
9                 Elem elem = get(cur);
10                cur = LinearList.this.next(cur);
11                return elem;
12            }
13        };
14    }
15 }
```